

# GLIFO

Tecnología de Automatización Geométrica

Especificacion tecnica completa del sistema de glifos BDGB: que es un glifo, sus fundamentos matematicos, su arquitectura de sistemas y su modelo de funcionamiento.

|               |                                   |
|---------------|-----------------------------------|
| Tecnologia    | BDGB — Binary Dynamics Grid Brain |
| Version       | 1.0 (2026-06-08)                  |
| Autor         | Juan Diego RGB                    |
| Lenguaje      | C99, Python 3                     |
| Clasificacion | Primo, Semilla, Comun             |
| Ramas         | Online, Local, Hibrida            |

# 1. Introduccion

Este documento define formalmente la tecnologia **Glifo** del sistema BDGB (Binary Dynamics Grid Brain). Un glifo es una unidad computacional atomica que manipula informacion usando herramientas como instrumentos para cumplir un flujo de trabajo (Sistema). Su logica es de orquestacion: decide que herramienta invocar, con que argumentos, en que orden, y como procesar el resultado para la siguiente etapa. Las herramientas (scripts, APIs, binarios) ejecutan las operaciones concretas; el glifo las orquesta para completar la tarea asignada.

El sistema Glifo se basa en una rejilla binaria 16x16 con dinamica Kaprekar, un motor semantico con hash index, busqueda hibrida, NLP integrado, cifrado de flujo propio (BDGB-Cipher v1) y aprendizaje por refuerzo. Todo el nucleo opera sin GPU, sin LLM, y sin dependencias externas en C.

Este documento esta dirigido a desarrolladores, arquitectos de software y curiosos que quieran entender como funciona BDGB Glifo por dentro, desde sus fundamentos matematicos hasta su implementacion concreta.

## 2. Que es un Glifo

Un **Glifo** es un nodo operativo dentro de un Sistema. Su funcion es manipular informacion para cumplir el flujo de trabajo que se le encarga. Para lograrlo, invoca herramientas como instrumentos: les pasa datos de entrada, recibe resultados, y decide el siguiente paso. El glifo no ejecuta la logica de dominio directamente, pero si orquesta el flujo completo usando las herramientas adecuadas en cada etapa.

### Analogia:

Imagina un chef en una cocina. El chef (glifo) no cultiva vegetales ni fabrica ollas: usa herramientas (cuchillos, horno, ingredientes) para preparar el plato. Decide que herramienta usar, cuando, y en que orden. El plato final es responsabilidad del chef, no de las herramientas.

### Naturaleza del Glifo:

| Propiedad            | Descripcion                                                                        |
|----------------------|------------------------------------------------------------------------------------|
| Orquestador          | Invoca herramientas, pasa datos, recibe resultados, decide el siguiente paso       |
| Manipula informacion | Toma datos de entrada, los transforma usando herramientas, produce resultados      |
| Orientado al flujo   | Su proposito es completar el workflow completo, no solo transportar                |
| Declarativo          | Su pipeline se define en JSON (glifosenilla.json)                                  |
| Portatil             | Usa herramientas del SO: funciona en Windows, Linux, macOS, Android                |
| Componible           | Varios glifos forman un pipeline; un sistema puede ser glifo de otro sistema mayor |
| Estado medible       | Cada glifo cuenta ejecuciones, exitos y fallos                                     |

### 2.1 Clasificacion de los Glifos

Los glifos se clasifican en tres tipos segun su naturaleza y funcion dentro del sistema:

| Clase   | Que es                                                                                                                             | Ejemplo                                     |
|---------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Primo   | El codigo original del que se derivan todos los demas glifos. Es el modelo base de todo glifo. Existe uno solo por implementacion. | std::gl::forma::código::glifo               |
| Semilla | El archivo glifosenilla.json que define y crea un sistema. Sin semilla no hay sistema. Es el glifo de arranque de trabajo.         | glifo_hay_sistema::Es::glifo::código::glifo |
| Comun   | Glifos operativos que ejecutan las tareas dentro de un sistema. Pueden ser nativos (C) o externos (Python, etc.).                  | primero::trabajo::C++                       |

### Subtipos de glifo comun:

| Subtipo | Descripcion                                                    | Ventaja                          | Desventaja                              |
|---------|----------------------------------------------------------------|----------------------------------|-----------------------------------------|
| Nativo  | Compilado en el binario BDGB (C). No requiere runtime externo. | Alto rendimiento y portabilidad. | Requiere compilar para cada plataforma. |
| Externo | Script (Python, bash, etc.) dentro de la carpeta del sistema.  | Facil de modificar y extender.   | Requiere runtime (Python, etc.)         |

## 3. Matematicas del Glifo

Todo el sistema BDGB se basa en una rejilla binaria de 16x16 nodos (256 nodos, 8 bits por nodo). Cada nodo tiene un valor numerico del que se derivan todas sus propiedades geometricas y dinamicas. Estas propiedades alimentan el motor semantico, la busqueda, el NLP y el cifrado.

### 3.1 La Rejilla Binaria (Grid)

La rejilla es una matriz de  $16 \times 16 = 256$  posiciones. Cada posición almacena un byte (8 bits, valor 0-255). Las coordenadas (x, y) se derivan directamente del ID del nodo:

```
Nodo ID = 0..255
x = ID & 0x0F (bits bajos, 0-15)
y = ID >> 4 (bits altos, 0-15)
```

La estructura en memoria del nodo es de 4 bytes: id\_bits (uint8), x (uint8), y (uint8), y un byte de relleno para alineación.

**Propiedades derivadas de cada nodo:**

| Propiedad        | Formula                         | Rango | Uso                     |
|------------------|---------------------------------|-------|-------------------------|
| Popcount         | bits = popcount(n)              | 0-8   | Densidad del nodo       |
| Simetria         | n == reverse_bits(n)            | 0/1   | Filtro de búsqueda      |
| Tipo geometrico  | esquina(x,y) / borde / interior | 0-2   | Clasificación espacial  |
| Radio            | distancia al centro (8,8)       | 0-12  | Proximidad al centro    |
| Clase dinamica   | punto_fijo / ciclo / atractor   | 0-2   | Comportamiento temporal |
| Pasos a atractor | iteraciones hasta converger     | 0-16  | Complejidad dinamica    |

### 3.2 Dinamica Kaprekar Binaria

Cada nodo tiene un sucesor calculado por la regla Kaprekar binaria:

1. Ordenar los bits del valor actual en orden ascendente (asc)
2. Ordenar los bits en orden descendente (desc)
3. Sucesor = desc - asc (en binario)
4. Repetir hasta alcanzar un punto fijo (atractor)

Por ejemplo: el nodo con valor 0x1B (00011011 en binario):

ascendente = 00000111 (7), descendente = 11100000 (224)

sucesor =  $224 - 7 = 217$  (11011001)

Este proceso se repite hasta que sucesor == nodo actual (punto fijo).

**Propiedades de la dinamica:** Todos los 256 nodos convergen a puntos fijos (no hay ciclos de longitud > 1). Los puntos fijos son valores cuyo binario en orden ascendente y descendente son iguales. Esto genera cuencas de atracción que son la base del motor de búsqueda.

### 3.3 Relacion con el Cifrado BDGB-Cipher

El mismo modelo matematico de la rejilla se utiliza para el cifrado de flujo BDGB-Cipher v1. El estado interno del cifrado son 4 semillas de 32 bits (128 bits total) que se mezclan mediante la función geometrica del nodo. La S-box se deriva de las propiedades del nodo BDGB calculadas en tiempo real. No utiliza ninguna biblioteca criptografica externa.

## 4. Sistema de Funcionamiento

Un **Sistema** es un flujo de trabajo con un propósito definido. Contiene glifos, relaciones entre ellos, un pipeline de ejecución y datos propios del sistema. Cada sistema tiene un archivo glifo maestro obligatorio llamado **glifosenilla.json** en su raíz.

### 4.1 Arquitectura Sistema > Glifo

La jerarquía es simple: un Sistema contiene uno o más Glifos. Un Glifo no existe fuera de un Sistema (aunque su código pueda crearse aparte, no se ejecuta hasta que se declara en un glifosenilla.json).

**Estructura de directorios:**

```
glifos/
registry.json <- registro maestro de sistemas y glifos
<sistema>/
glifosenilla.json <- SEMILLA: define todo el sistema
glifos/
<glifo>/
glifo.json <- config individual del glifo
...codigo del glifo...
```

### 4.2 El Archivo glifosenilla.json (La Semilla)

Este archivo es la autoridad del sistema. Define que glifos existen, como se relacionan, en que orden se ejecutan, y los datos propios del sistema. Sin este archivo no hay sistema.

| Campo         | Descripcion                                           |
|---------------|-------------------------------------------------------|
| id            | Identificador unico del sistema                       |
| nombre        | Nombre humano                                         |
| tipo          | Siempre 'sistema'                                     |
| rama          | online, local, o hibrida                              |
| glifos[]      | Lista de glifos del sistema (id, nombre, tipo, entry) |
| relaciones[]  | Conexiones entre glifos (de, a, tipo, flujo)          |
| pipeline[]    | Orden de ejecucion con dependencias                   |
| datos_sistema | Datos personalizados segun la tarea del sistema       |
| metricas      | Contadores de ejecucion                               |

### 4.3 Pipeline y Relaciones

El **pipeline** define el orden de ejecucion de los glifos. Cada paso tiene un orden, un glifo asignado, una accion, argumentos opcionales y dependencias. Las **relaciones** definen conexiones conceptuales entre glifos (ejecucion, dependencia, datos compartidos).

#### Ejemplo de pipeline:

```
Paso 1: glifo 'primo' - fetch RSS + reporte + inyeccion en BDGB
Paso 2: glifo 'trend-tracker' --daily (depende de primo)
Paso 3: glifo 'trend-tracker' --weekly (cada 7 dias, depende de primo)
```

### 4.4 Ramas de Sistemas

Cada sistema pertenece a una de tres ramas segun el origen de sus herramientas:

| Rama    | Herramientas                                                                                                              | Ejemplos                                            |
|---------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Online  | Servicios web, APIs externas. Requieren conexiones a internet                                                             | Google Trends, YouTube API, OpenAI, scraping web    |
| Local   | Corren 100% en el dispositivo. No requieren internet                                                                      | Scripts Python, binarios C, herramientas CLI del SO |
| Hibrida | Combinan herramientas online y locales. Lo mas comun es el scraping web la practica es el procesamiento local + API cloud |                                                     |

## 5. Motor Semantico

El motor semantico asocia nodos de la rejilla con conceptos numericos mediante un sistema de hash index. Cada nodo puede tener multiples conceptos, y los conceptos se relacionan entre si mediante un grafo de aristas.

### 5.1 Hash Index de Conceptos

La semantica usa un hash de 256 buckets (0-255) derivado del concepto ID:

```
hash = (concept_id ^ (concept_id >> 4)) & 0xFF
```

Cada bucket tiene una cadena maxima de 16 entradas. En la practica, con menos de 50 conceptos, las colisiones son minimas y la busqueda es O(1).

### 5.2 Grafo de Conceptos

Los conceptos se relacionan entre si mediante aristas con peso y tipo (definicion, ejemplo, metafora, funcion, causa). El grafo usa el mismo sistema de hash index para busquedas rapidas desde cualquier concepto hacia sus vecinos.

### 5.3 Busqueda Hibrida

La busqueda combina cuatro estrategias:

1. **Semantica:** busca nodos asociados a un concepto especifico
2. **Propiedades:** filtra por simetria, densidad, tipo geometrico, etc.
3. **Atractor:** busca nodos cerca de un atractor especifico
4. **NLP:** tokeniza texto libre y mapea a conceptos + predicados

Los resultados se refuerzan con aprendizaje (incremento de uso) y se ordenan por puntuacion descendente.

## 6. NLP (Procesamiento de Lenguaje Natural)

El modulo NLP permite consultar el sistema BDGB usando texto libre. Convierte palabras en terminos que mapean a conceptos y predicados dentro del sistema.

### 6.1 Tokenizacion

El texto de entrada se divide en tokens (palabras de 3-20 caracteres, ignorando puntuacion y espacios). Cada token se busca en un diccionario de terminos que puede crecer dinamicamente.

### 6.2 Sistema de Predicados

Algunos terminos NO mapean a conceptos sino a predicados: funciones que filtran nodos por propiedades. Por ejemplo:

| Termino   | Predicado                   | Filtro           |
|-----------|-----------------------------|------------------|
| simetrico | simetria == 1               | Nodos simetricos |
| denso     | densidad >= 3               | Nodos densos     |
| interior  | tipo_geom == INTERIOR       | Nodos interiores |
| esquina   | tipo_geom == ESQUINA        | Esquinas         |
| borde     | tipo_geom == BORDE          | Bordes           |
| estable   | clase_dinamica == ATTRACTOR | Puntos fijos     |

### 6.3 Aprendizaje Dinamico

El diccionario NLP no es estatico. Cuando un glifo inyecta datos en BDGB, el modulo `nlp_learn_from_text()` extrae automaticamente nuevos terminos del texto de entrada, los asocia a conceptos numericos y los persiste en disco. El sistema comenzo con ~20 terminos fijos y ha crecido a 55+ terminos aprendidos desde glifos externos.

## 7. BDGB-Cipher v1

BDGB-Cipher es un cifrado de flujo (stream cipher) disenado completamente sobre el modelo matematico BDGB, sin utilizar ninguna biblioteca criptografica externa. Es la demostracion de que el mismo modelo que genera propiedades geometricas puede servir para cifrar datos.

**Caracteristicas del cifrado:**

| Propiedad           | Valor                                                    |
|---------------------|----------------------------------------------------------|
| Estado interno      | 4 x uint32_t = 128 bits + IV 8 bytes + contador 64 bytes |
| S-box               | Geometrica: propiedades del nodo BDGB en tiempo real     |
| Derivacion de clave | mix32() con 8 rondas, retroalimentacion cruzada          |
| Keystream           | 1 byte por ciclo, estado + contador                      |
| Formato archivo     | Magic 'BDGB' + version 0x01 + IV 8 B + ciphertext        |
| Dependencias        | CERO (no OpenSSL, no libcrypto)                          |
| Espacio de claves   | 2 <sup>128</sup>                                         |

El cifrado demuestra la versatilidad del modelo matematico BDGB: lo que sirve para buscar informacion semanticamente tambien sirve para protegerla. Es la misma maquina con dos propositos distintos.

## 8. Rendimiento y Portabilidad

El sistema Glifo esta disenado desde cero para ser portatil y eficiente.

**Portabilidad:**

La logica de orquestacion del glifo (decidir que herramienta invocar, con que argumentos, en que orden) es inherentemente portable: funciona identico en Windows, Linux, macOS, Android, iOS, tablets y cualquier sistema con un sistema de archivos. El nucleo C compila con C99 estandar en cualquier plataforma. Las herramientas externas pueden requerir runtimes especificos (Python, etc.), pero el glifo que las orquesta no.

## Rendimiento:

| Metrica            | Valor                              |
|--------------------|------------------------------------|
| Binario            | ~150 KB (bdgb.exe)                 |
| Tiempo de inicio   | < 10 ms                            |
| Busqueda semantica | O(1) con hash index                |
| Busqueda hibrida   | O(n) sobre 256 nodos               |
| Tokenizacion NLP   | O(m) sobre m caracteres de entrada |
| Cifrado/descifrado | ~2 MB/s en CPU moderna             |
| Memoria en reposo  | ~300 KB                            |

## Sin dependencias:

El nucleo C (incluyendo el cifrado) tiene CERO dependencias externas. No necesita OpenSSL, libcurl, libxml, ni ninguna biblioteca de terceros. El unico requisito para compilar es un compilador C99 y CMake. Las herramientas externas (Python) son opcionales y se usan solo para glifos externos; el nucleo funciona sin ellas.

## 9. Ramas de Sistemas en Detalle

---

### 9.1 Rama Online

Los sistemas de la rama **online** utilizan herramientas que dependen de servicios web o APIs externas. Ejemplos: Google Trends RSS, YouTube API, OpenAI API, scraping de paginas web. Estos sistemas requieren conexion a internet y manejan autenticacion (API keys, tokens).

El glifo orquesta las herramientas online: invoca la herramienta adecuada (ej: trend\_tracker.py), recibe el resultado, y decide si pasarlo a la siguiente etapa del pipeline, combinarlo con otros datos, o reprocesarlo. El glifo no hace la llamada HTTP directamente, pero es quien decide que llamada hacer y cuando.

### 9.2 Rama Local

Los sistemas de la rama **local** funcionan completamente sin internet. Todas las herramientas corren en el dispositivo: scripts Python, binarios C, herramientas CLI del sistema operativo. Los datos se generan y procesan localmente.

Un sistema local tipico: el glifo invoca un escaner local que produce un JSON, toma ese JSON y lo pasa a una herramienta de analisis, recibe el reporte, y lo entrega a una herramienta de visualizacion. El glifo decide el flujo completo. Todo offline.

### 9.3 Rama Hibrida

Los sistemas de la rama **hibrida** combinan herramientas online y locales. Es la rama mas comun en la practica. Por ejemplo: un scraper web (online) descarga datos, un procesador local (Python) los analiza, y el resultado se sube a una API cloud (online).

El glifo abstrae esta complejidad: para el, cada herramienta es un modulo con entradas y salidas. No le importa si la herramienta hizo una llamada HTTP o un calculo local. Lo que importa es que el flujo completo se complete correctamente.

## 10. Conclusion

---

La tecnologia Glifo de BDGB es un enfoque unico para la automatizacion de flujos de trabajo. Su innovacion principal es la orquestacion declarativa: el glifo manipula informacion usando herramientas como instrumentos, y su logica se define en JSON. Esto hace que los glifos sean portatiles, componibles y faciles de encadenar en pipelines complejos.

Los fundamentos matematicos (rejilla binaria, dinamica Kaprekar, hash index semantico) proporcionan una base solida y predecible para el sistema, sin depender de bibliotecas externas, GPUs o modelos de lenguaje masivos. El mismo modelo matematico sirve para buscar, aprender y cifrar.

La clasificacion en tres ramas (Online, Local, Hibrida) organiza los sistemas segun la naturaleza de sus herramientas, y las tres clases de glifos (Primo, Semilla, Comun) establecen una jerarquia conceptual clara desde la implementacion base hasta los nodos operativos.

